BeaconKit 아키텍처 분석 - 05. 블록 처리 및 검증 프로세스

1. 블록 처리 개요

BeaconKit의 블록 처리는 **ProcessProposal**과 **FinalizeBlock** 두 단계로 나뉩니다. ProcessProposal에서는 블록을 검증하고, FinalizeBlock에서는 실제 상태 변경을 수행합니다. 이는 CometBFT의 ABCI++ 패턴을 따릅니다.

1.1 핵심 흐름

```
제안된 블록 수신 → ProcessProposal (검증) → 합의 투표 → FinalizeBlock (실행) → Commi
```

1.2 상태 관리 전략

- **ProcessProposal**: 임시 상태에서 검증, 커밋하지 않음
- **FinalizeBlock**: 실제 상태 변경 수행
- **Commit**: 변경사항을 디스크에 영구 저장

2. ProcessProposal 상세 분석

2.1 CometBFT Service의 processProposal

```
func (s *Service) processProposal(
ctx context.Context,
```

```
req *cmtabci.ProcessProposalRequest,
) (*cmtabci.ProcessProposalResponse, error) {
startTime := time.Now()
defer  s.telemetrySink.MeasureSince("beacon_kit.runtime.process_proposal_duratio

// STEP 1: 높이 검증
if  req.Height < 1 {
return  nil, fmt.Errorf("processProposal at height %v: %w", req.Height, errInval
}

// STEP 2: 임시 상태 생성 (이전 블록 상태 기반)
processProposalState := s.resetState(ctx)
processProposalState.SetContext(s.getContextForProposal(processProposalState.Con

// STEP 3: 블록체인 서비스에서 실제 검증 수행
valUpdates, err := s.Blockchain.ProcessProposal(
processProposalState.Context(),
req,
s.nodeAddress[:],
)

if  err != nil {
s.logger.Error("failed to process proposal", "height", req.Height, "err", err)
```

```
return &cmtabci.ProcessProposalResponse{Status: cmtabci.PROCESS_PROPOSAL_STATUS_
}

// STEP 4: 검증 성공 시 상태 캐싱 (재사용을 위해)
if  cache.IsStateCachingActive(s.chainSpec, req.Height) {
stateHash := string(req.Hash)
toCache := &cache.Element{
State: processProposalState,
ValUpdates: valUpdates,
}
s.cachedStates.Add(stateHash, toCache)
}

return &cmtabci.ProcessProposalResponse{Status: cmtabci.PROCESS_PROPOSAL_STATUS_
}
```

2.2 Blockchain Service의 ProcessProposal

```
func (s *Service) ProcessProposal(
ctx sdk.Context,
req *cmtabci.ProcessProposalRequest,
thisNodeAddress []byte,
```

```go
) (transition.ValidatorUpdates, error) {

// STEP 1: 블록 및 블롭 디코딩

signedBlk, sidecars, err := s.ParseBeaconBlock(req)

if  err != nil {

s.logger.Error("Failed to decode block and blobs", "error", err)

return  nil, fmt.Errorf("failed to decode block and blobs: %w", err)

}

blk := signedBlk.GetBeaconBlock()



// STEP 2: 포크 버전 검증

forkVersion := s.chainSpec.ActiveForkVersionForTimestamp(math.U64(req.GetTime().

blkVersion := s.chainSpec.ActiveForkVersionForTimestamp(blk.GetTimestamp())

if !version.Equals(blkVersion, forkVersion) {

return  nil, fmt.Errorf("CometBFT version %v, BeaconBlock version %v: %w",

forkVersion, blkVersion, ErrVersionMismatch)

}



// STEP 3: 블롭 커밋먼트 수 검증

blobKzgCommitments := blk.GetBody().GetBlobKzgCommitments()

numCommitments := len(blobKzgCommitments)

if  numCommitments != len(sidecars) {

return  nil, fmt.Errorf("expected %d sidecars, got %d: %w",

numCommitments, len(sidecars), ErrSidecarCommitmentMismatch)
```

```
}
if uint64(numCommitments) > s.chainSpec.MaxBlobsPerBlock() {
return nil, fmt.Errorf("expected less than %d sidecars, got %d: %w",
s.chainSpec.MaxBlobsPerBlock(), numCommitments, core.ErrExceedsBlockBlobLimit)
}

// STEP 4: 블록 서명 검증
blkSignature := signedBlk.GetSignature()
for i, sidecar := range sidecars {
sidecarSignature := sidecar.GetSignature()
if !bytes.Equal(blkSignature[:], sidecarSignature[:]) {
return nil, fmt.Errorf("%w, idx: %d", ErrSidecarSignatureMismatch, i)
}
}
err = s.VerifyIncomingBlockSignature(ctx, blk, signedBlk.GetSignature())
if err != nil {
return nil, err
}

// STEP 5: 블롭 사이드카 검증 (KZG 증명)
if numCommitments > 0 {
err = s.VerifyIncomingBlobSidecars(ctx, sidecars, blk.GetHeader(), blobKzgCommit
if err != nil {
```

```
s.logger.Error("failed to verify incoming blob sidecars", "error", err)
return  nil, err
}
}

// STEP 6: 블록 상태 전환 검증
consensusBlk := types.NewConsensusBlock(blk, req.GetProposerAddress(), req.GetTi
valUpdates, err := s.VerifyIncomingBlock(ctx, consensusBlk, bytes.Equal(thisNode
if  err != nil {
s.logger.Error("failed to verify incoming block", "error", err)
return  nil, err
}

return  valUpdates.CanonicalSort(), nil
}
```

2.3 검증 단계별 상세 분석

2.3.1 구조적 검증

```
// 블록 디코딩 및 기본 구조 검증
signedBlk, sidecars, err := s.ParseBeaconBlock(req)

// 포크 버전 일치성 검증
```

```go
if !version.Equals(blkVersion, forkVersion) {
return  nil, ErrVersionMismatch
}

// 블롭 수 제한 검증
if  uint64(numCommitments) > s.chainSpec.MaxBlobsPerBlock() {
return  nil, core.ErrExceedsBlockBlobLimit
}
```

2.3.2 암호학적 검증

```go
// 블록 서명 검증
func (s *Service) VerifyIncomingBlockSignature(
ctx context.Context,
beaconBlk *ctypes.BeaconBlock,
signature crypto.BLSSignature,
) error {
signatureVerifierFn, err := s.stateProcessor.GetSignatureVerifierFn(
s.storageBackend.StateFromContext(ctx),
)
if  err != nil {
return  errors.New("failed to create block signature verifier")
}
```

```go
return  signatureVerifierFn(beaconBlk, signature)
}


// 블롭 KZG 증명 검증
func (s *Service) VerifyIncomingBlobSidecars(
ctx context.Context,
sidecars datypes.BlobSidecars,
blkHeader *ctypes.BeaconBlockHeader,
kzgCommitments eip4844.KZGCommitments[common.ExecutionHash],
) error {
return  s.blobProcessor.VerifySidecars(ctx, sidecars, blkHeader, kzgCommitments)
}
```

2.3.3 상태 전환 검증

```go
func (s *Service) verifyStateRoot(
ctx context.Context,
st *statedb.StateDB,
blk *types.ConsensusBlock,
) (transition.ValidatorUpdates, error) {
// 상태 캐싱 활성화 여부 확인
isCacheActive := cache.IsStateCachingActive(s.chainSpec, blk.GetBeaconBlock().Ge
// 전환 컨텍스트 생성 (검증 모드)
txCtx := transition.NewTransitionCtx(ctx, blk.GetConsensusTime(), blk.GetPropose
```

```
WithVerifyPayload(true). // 실행 페이로드 검증
WithVerifyRandao(true). // RANDAO 검증
WithVerifyResult(true). // 결과 검증
WithMeterGas(isCacheActive) // 가스 측정 (캐시 활성화 시)


// 상태 전환 실행 및 검증
valUpdates, err := s.stateProcessor.Transition(txCtx, st, blk.GetBeaconBlock())
return valUpdates, err
}
```

3. FinalizeBlock 상세 분석

3.1 CometBFT Service의 finalizeBlock

```
func (s *Service) finalizeBlock(
ctx context.Context,
req *cmtabci.FinalizeBlockRequest,
) (*cmtabci.FinalizeBlockResponse, error) {
// STEP 1: 높이 검증
if err := s.validateFinalizeBlockHeight(req); err != nil {
return nil, err
}
```

```
// STEP 2: 캐시된 상태 사용 시도 (ProcessProposal에서 캐시된 것)
if  cache.IsStateCachingActive(s.chainSpec, req.Height) {
hash := string(req.Hash)
if  cached, err := s.cachedStates.Get(hash);  err == nil {
// 캐시된 상태로 빠른 최종화
finalState, valUpdates := cached.State, cached.ValUpdates
// 사이드카 최종화
if  err = s.Blockchain.FinalizeSidecars(finalState.Context(), req.SyncingToHeigh
return  nil, fmt.Errorf("failed finalizing sidecars: %w", err)
}
// 후처리 작업
if  err = s.Blockchain.PostFinalizeBlockOps(finalState.Context(), blk);  err !=
return  nil, fmt.Errorf("finalize block: failed post finalize block ops: %w", er
}
return  s.calculateFinalizeBlockResponse(req, cached.ValUpdates)
}
}

// STEP 3: 캐시 실패 시 전체 실행
finalState := s.resetState(ctx)
finalState.SetContext(s.getContextForProposal(finalState.Context(), req.Height))

// 실제 블록 최종화 수행
```

```go
valUpdates, err := s.Blockchain.FinalizeBlock(finalState.Context(), req)
if err != nil {
return nil, err
}

// 최종 상태를 커밋 준비 상태로 마킹
if err = s.cachedStates.MarkAsFinal(string(req.Hash)); err != nil {
return nil, fmt.Errorf("failed to mark state as final: %w", err)
}

return s.calculateFinalizeBlockResponse(req, valUpdates)
}
```

3.2 Blockchain Service의 FinalizeBlock

```go
func (s *Service) FinalizeBlock(
ctx sdk.Context,
req *cmtabci.FinalizeBlockRequest,
) (transition.ValidatorUpdates, error) {
// STEP 1: 블록 및 블롭 디코딩
signedBlk, blobs, err := s.ParseBeaconBlock(req)
if err != nil {
```

```
s.logger.Error("Failed to decode block and blobs", "error", err)
return  nil, fmt.Errorf("failed to decode block and blobs: %w", err)
}
blk := signedBlk.GetBeaconBlock()
st := s.storageBackend.StateFromContext(ctx)


// STEP 2: 시작 시 강제 동기화 (한 번만)
var  finalizeErr  error
s.forceStartupSyncOnce.Do(func() {
var  parentProposerPubkey *crypto.BLSPubkey
parentProposerPubkey, finalizeErr = st.ParentProposerPubkey(blk.GetTimestamp())
if  finalizeErr != nil {
finalizeErr = fmt.Errorf("force sync upon finalize: failed retrieving parent pro
} else {
finalizeErr = s.forceSyncUponFinalize(ctx, blk, parentProposerPubkey)
}
})
if  finalizeErr != nil {
return  nil, finalizeErr
}


// STEP 3: 사이드카 최종화 (블록이 사이드카 가용성을 확인)
if  err = s.FinalizeSidecars(ctx, req.SyncingToHeight, blk, blobs);  err != nil
```

```go
return  nil, fmt.Errorf("failed finalizing sidecars: %w", err)
}

// STEP 4: 블록 최종화
consensusBlk := types.NewConsensusBlock(blk, req.GetProposerAddress(), req.GetTi
valUpdates, err := s.finalizeBeaconBlock(ctx, st, consensusBlk)
if  err != nil {
s.logger.Error("Failed to process verified beacon block", "error", err)
return  nil, err
}

// STEP 5: 후처리 작업
return  valUpdates, s.PostFinalizeBlockOps(ctx, blk)
}
```

3.3 사이드카 최종화

```go
func (s *Service) FinalizeSidecars(
ctx sdk.Context,
syncingToHeight  int64,
blk *ctypes.BeaconBlock,
blobs datypes.BlobSidecars,
) error {
```

```go
// DA 기간 내에 있는지 확인
if s.chainSpec.WithinDAPeriod(blk.GetSlot(), math.Slot(syncingToHeight)) {
// 블롭 사이드카 처리
err := s.blobProcessor.ProcessSidecars(
s.storageBackend.AvailabilityStore(),
blobs,
)
if err != nil {
s.logger.Error("Failed to process blob sidecars", "error", err)
return fmt.Errorf("failed to process blob sidecars: %w", err)
}

// 데이터 가용성 확인
if !s.storageBackend.AvailabilityStore().IsDataAvailable(ctx, blk.GetSlot(), blk
return ErrDataNotAvailable
}
return nil
}

// DA 기간 외부에서는 블롭 처리 스킵
if len(blobs) > 0 {
s.logger.Info("Skipping blob processing outside of Data Availability Period",
"slot", blk.GetSlot().Base10(), "head", syncingToHeight)
```

```
}
return nil
}
```

3.4 상태 전환 실행

```
func (s *Service) executeStateTransition(
ctx context.Context,
st *statedb.StateDB,
blk *types.ConsensusBlock,
) (transition.ValidatorUpdates, error) {
startTime := time.Now()
defer s.metrics.measureStateTransitionDuration(startTime)

// 전환 컨텍스트 생성 (실행 모드)
txCtx := transition.NewTransitionCtx(ctx, blk.GetConsensusTime(), blk.GetPropose
WithVerifyPayload(true). // 실행 페이로드 검증 (동기화 상태에 관계없이)
WithVerifyRandao(false). // RANDAO 검증 스킵 (ProcessProposal에서 이미 검증)
WithVerifyResult(false). // 결과 검증 스킵 (이미 합의됨)
WithMeterGas(true) // 가스 측정 활성화

return s.stateProcessor.Transition(txCtx, st, blk.GetBeaconBlock())
```

```
}
```

3.5 후처리 작업

```
func (s *Service) PostFinalizeBlockOps(ctx sdk.Context, blk *ctypes.BeaconBlock)
st := s.storageBackend.StateFromContext(ctx)

// STEP 1: 예치 데이터 가져오기 및 저장
blockNum := blk.GetBody().GetExecutionPayload().GetNumber()
s.depositFetcher(ctx, blockNum)

// STEP 2: 블록을 KV 저장소에 저장
slot := blk.GetSlot()
if err := s.storageBackend.BlockStore().Set(blk); err != nil {
s.logger.Error("failed to store block", "slot", slot, "error", err)
return err
}

// STEP 3: 가용성 및 예치 저장소 정리
if err := s.processPruning(ctx, blk); err != nil {
s.logger.Error("failed to processPruning", "error", err)
}
```

```go
// STEP 4: 실행 엔진에 FCU 업데이트 전송
if err := s.sendPostBlockFCU(ctx, st); err != nil {
return fmt.Errorf("sendPostBlockFCU failed: %w", err)
}

return nil
}
```

4. 상태 캐싱 메커니즘

4.1 캐시 구조

```go
type States interface {
// Add는 상태를 캐시에 추가
Add(hash string, element *Element)
// Get은 캐시된 상태를 검색
Get(hash string) (*Element, error)
// MarkAsFinal은 캐시된 상태 중 하나를 최종화 대상으로 마킹
MarkAsFinal(hash string) error
// GetFinal은 최종화로 마킹된 상태를 검색
GetFinal() (string, *Element, error)
}
```

```go
type Element struct {
State *cache.State // 캐시된 상태
ValUpdates transition.ValidatorUpdates // 검증자 업데이트
}
```

4.2 캐시 활용 전략

ProcessProposal에서 캐싱:

```go
if cache.IsStateCachingActive(s.chainSpec, req.Height) {
stateHash := string(req.Hash)
toCache := &cache.Element{
State: processProposalState,
ValUpdates: valUpdates,
}
s.cachedStates.Add(stateHash, toCache)
}
```

FinalizeBlock에서 재사용:

```go
if cached, err := s.cachedStates.Get(hash); err == nil {
// 캐시된 상태로 빠른 최종화
finalState, valUpdates := cached.State, cached.ValUpdates
// ... 사이드카 및 후처리만 수행
```

```
}
```

5. 성능 최적화

5.1 검증 단계 최적화

조기 실패 전략:

```
// 1. 구조적 검증 먼저 (빠름)
if numCommitments != len(sidecars) {
return nil, ErrSidecarCommitmentMismatch
}

// 2. 암호학적 검증 (중간)
err = s.VerifyIncomingBlockSignature(ctx, blk, signature)

// 3. 상태 전환 검증 마지막 (느림)
valUpdates, err := s.verifyStateRoot(ctx, st, blk)
```

병렬 처리:

```
// 블롭 KZG 증명 병렬 검증
err = s.blobProcessor.VerifySidecars(ctx, sidecars, blkHeader, kzgCommitments)
```

5.2 상태 캐싱 최적화

캐시 적중률 향상:

- ProcessProposal에서 검증된 상태를 FinalizeBlock에서 재사용
- 중복 상태 전환 계산 방지
- 메모리 사용량과 성능의 균형

5.3 메트릭 기반 모니터링

```
startTime := time.Now()
defer s.telemetrySink.MeasureSince("beacon_kit.runtime.process_proposal_duratio
defer s.metrics.measureStateTransitionDuration(startTime)
```

6. 오류 처리 전략

6.1 검증 실패 처리

```
if err != nil {
s.logger.Error("failed to verify incoming block", "error", err)
return &cmtabci.ProcessProposalResponse{Status: cmtabci.PROCESS_PROPOSAL_STATUS_
}
```

특징:

- 검증 실패 시 REJECT 반환 (오류가 아님)
- 새로운 라운드에서 다른 제안자가 시도
- 네트워크 전체 중단 방지

6.2 최종화 실패 처리

```
valUpdates, err := s.Blockchain.FinalizeBlock(finalState.Context(), req)
if err != nil {
return nil, err // 치명적 오류로 처리
}
```

특징:

- FinalizeBlock 실패는 치명적 오류
- 합의가 이미 이루어진 블록의 실행 실패
- 노드 패닉 및 재시작 필요

7. 설계 원칙

7.1 검증과 실행의 분리

- ProcessProposal: 검증만, 상태 변경 없음
- FinalizeBlock: 실제 상태 변경 수행

7.2 성능과 안전성의 균형

- 캐시를 통한 성능 향상
- 검증 단계의 체계적 구성
- 조기 실패를 통한 자원 절약

7.3 결정론적 처리

- 모든 노드에서 동일한 결과

- 비결정적 요소 제거
- 재현 가능한 상태 전환

이 블록 처리 및 검증 프로세스는 BeaconKit이 안전하고 효율적으로 블록을 처리하면서도 CometBFT의 합의 요구사항을 만족시키는 핵심 메커니즘입니다.